

Other Neural Networks

Sequential prediction tasks

Many applications in real world require the **computation of data as sequences**

- Variable lengths **in time**
- Variable lengths **in space**

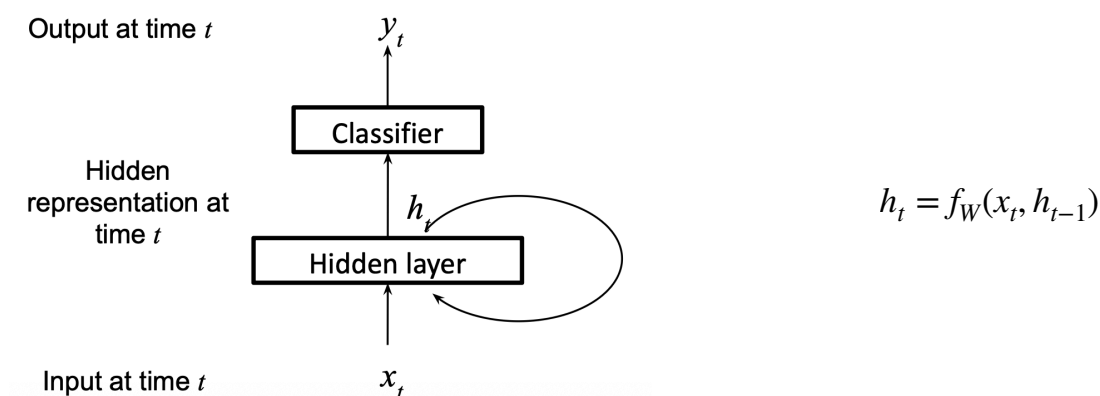
Examples are:

- Document classification
- Sentiment analysis
- Image captioning
- Video frame prediction

RNNs - Recurrent Neural Networks

To manage the use of sequences, **Recurrent Neural Networks** (= **RNNs**) are **used**

- They use **explicit cycles in their architecture**



At each time step t the net:

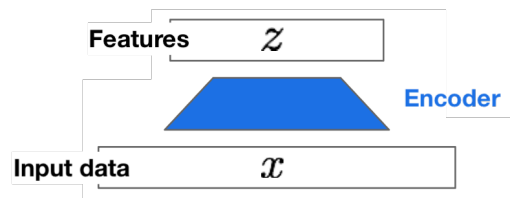
- Receives the **input** x_t
- **Updates** the **hidden representation** h_t
 - Is the **historical memory** of the net
 - Is **computed in function** of:
 - The **input** x_t
 - The **previous state** h_{t-1}
- The **classifier predicts** the value y_t

💡 **RNNs** can be **trained** with **backpropagation**

Autoencoders

They are an **unsupervised approach to learn a lower-dimensional feature representation**

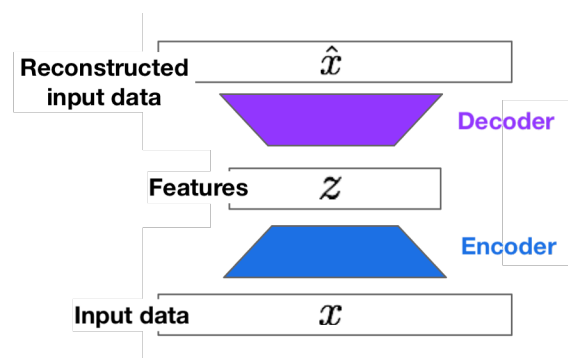
- From unlabeled training data
- **Features** should **capture meaningful factors of variation** in data



How to learn such a feature representation?

- The **features obtained** can be **used to reconstruct the original data**

Architecture



An **autoencoder** is structurally designed as a **clepsydra**:

- **Input data** x as the **base**
- **Features** z as the **chokepoint**
- **Reconstructed data** $\hat{x}$ as the **topping**

2 functional components:

- **Encoder**
 - **Receives** the **input data** x
 - **Compresses** the **input space** and **returns the features** z
- **Decoder**
 - **Receives** the **features** z as input
 - **Reconstructs** the **original data** and **returns** $\hat{x}$

Training

💡 The **encoder** is lately **implement as a deep CNN with ReLU** activation functions

- Backpropagation is used in learning

The **loss function** is defined as the **difference between** the **original data** x and the **reconstructed data** $\hat{x}$:

$$L = \|x - \hat{x}\|^2$$

Initializing a supervised model

After training the **decoder** is **removed**

The **encoder**:

- Has **learnt the optimal weights** to solve the problem
- Can be **used to initialize a supervised model**
 - Performs dimensionality reduction before a classifier is activated

